

SHACL Validation under the Well-founded Semantics

21st International Conference on Principles of Knowledge Representation and Reasoning
November 8th, 2024 | Hanoi, Vietnam

Cem Okulmus¹ Mantas Šimkus²

¹ Umeå University, Sweden

² TU Wien, Austria



UMEÅ UNIVERSITY



SHACL - SHApes Constraint Language

Background

- SHACL is a schema language for RDF datasets.
- W3C Recommendation in 2017.
- Features: support for detecting “shapes”, via modular set of constraints, making use of regular path queries to detect connected structures inside RDF datasets.

(not to be confused with shackle(s))



sh-handcuffs-0000 - 120070020

SHACL - SHAPes Constraint Language

Background

- SHACL is a schema language for RDF datasets.
- W3C Recommendation in 2017.
- Features: support for detecting “shapes”, via modular set of constraints, making use of regular path queries to detect connected structures inside RDF datasets.

```
:MusicianShape a sh:NodeShape;  
  sh:targetClass dbo:Person;  
  sh:node :PersonShape;  
  sh:property [  
    sh:path dbo:instrument;  
    sh:minCount 1;  
  ] .
```

TargetSet $\subseteq$ *ShapeSet*

SHACL - SHApes Constraint Language

Background

- SHACL is a schema language for RDF datasets.
- W3C Recommendation in 2017.
- Features: support for detecting “shapes”, via modular set of constraints, making use of regular path queries to detect connected structures inside RDF datasets.

(simplified logical notation + ignoring targetting)

$$\textit{MusicianShape} \leftarrow \textit{PersonShape} \wedge (\exists \textit{instrument}.\top)$$

SHACL and its connection to Description Logic

- Despite some initial confusion on this, SHACL can be seen as a Description Logic (DL). Some might even call it a DL “in disguise”¹.
- Following that work, a SHACL document is sets of triples $(\langle s, \varphi_s, q_s \rangle)$ consisting of three elements: 1) a shape names s , 2) a shape expression φ_s and target queries q_s , where we have that

$$s \equiv \varphi_s \quad q_s \sqsubseteq \varphi_s$$

- Hence, SHACL can be seen as a DL with a TBox consisting of pairs of constraints for each shape. One important distinction from regular DLs: **a partial closed world semantics**. It is closed on relations, but open on the domain.

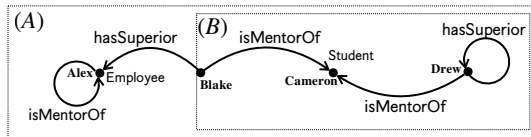
¹SHACL: A Description Logic in Disguise, B. Bogaerts, M. Jakubowski, J. Van den Bussche, LPNMR 2022

The Problem With SHACL

Recursion - To Be, Or Not To Be?

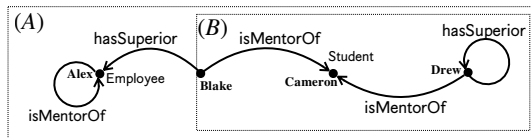
- W3C Recommendation allows for recursion, but does not provide a clear semantics, it is left as an implementation detail and indeed does not have to be supported.
- This led to a number of proposals for the semantics of SHACL under recursion, all of which have certain drawbacks (as we shall explain)

Running Example on problems with Recursion (1/4)



$ProfShape \leftarrow EmplShape \wedge (\exists isMentorOf.StudShape)$
 $EmplShape \leftarrow Employee \vee \exists hasSuperior.EmplShape$
 $StudShape \leftarrow Student \vee (EmplShape \wedge \neg ProfShape)$

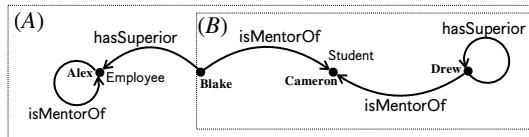
Running Example on problems with Recursion (2/4)


$$ProfShape \leftarrow EmplShape \wedge (\exists isMentorOf.StudShape)$$
$$EmplShape \leftarrow Employee \vee \exists hasSuperior.EmplShape$$
$$StudShape \leftarrow Student \vee (EmplShape \wedge \neg ProfShape)$$

Validation Rules

Identify sets of nodes, associated with a name (cf. concept expressions in Description Logics)

Running Example on problems with Recursion (3/4)



$ProfShape \leftarrow EmplShape \wedge (\exists isMentorOf.StudShape)$

$EmplShape \leftarrow Employee \vee \exists hasSuperior.EmplShape$

$StudShape \leftarrow Student \vee (EmplShape \wedge \neg ProfShape)$

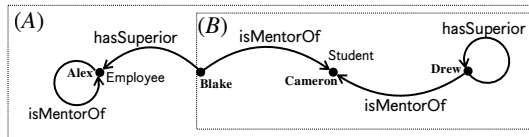
$\mathcal{T}_1 = \{(Blake, ProfShape), (Cameron, StudShape)\}$

2-valued supported semantics²

- Goal: find assignment of shape names to nodes s.t. : (i) a node o is assigned a shape name s iff o satisfies the shape expression associated to s , and (ii) the target specification is respected.
- Problem: KG (A) does not permit such an assignment. The problem is the node Alex: whether we assign to it $ProfShape$ or $\neg ProfShape$, it leads to a contradiction.

²Validating Shacl Constraints over a Sparql Endpoint, J. Corman, F. Florenzano, J.L.Reutter, O. Savkovic, ISWC 2019

Running Example on problems with Recursion (4/4)



$ProfShape \leftarrow EmplShape \wedge (\exists isMentorOf.StudShape)$

$EmplShape \leftarrow Employee \vee \exists hasSuperior.EmplShape$

$StudShape \leftarrow Student \vee (EmplShape \wedge \neg ProfShape)$

$\mathcal{T}_1 = \{(Blake, ProfShape), (Cameron, StudShape)\}$

3-valued supported semantics³

- As before, but we relax requirement on totality: not every node needs to be assigned a shape if it matches the shape expression (“unknown” value).
- Still Problematic: For subset $KG(B)$ we get the assignment $\{ProfShape(Blake), StudShape(Cameron), ProfShape(Drew), EmplShape(Drew)\}$. Why is Drew assigned *EmplShape* ? Because they supervise themselves, and *Drew* is assigned to be *EmplShape*. **Circular reasoning!**

³Validating Shacl Constraints over a Sparql Endpoint, J. Corman, F. Florenzano, J.L.Reutter, O. Savkovic, ISWC 2019

Brief primer on logic programming

Logic Program

We have a (countably infinite) set of propositional atoms N_{pa} , called *positive literals*. A *negative literal* is an expression $\neg a$, where $a \in N_{pa}$. A rule ρ is an expression of the form

$$\rho : h \leftarrow L_1, \dots, L_n \quad n \geq 0$$

where $h \in N_{pa}$ and L_i is a positive or negative literal. A logic program is a finite set of rules. We use the notation: $head(\rho) = h$ and $body(\rho) = \{L_1, \dots, L_n\}$.

3-Valued Interpretation

An *interpretation* I is a set of positive or negative literals, such that $\{a, \neg a\} \not\subseteq I$ for any $a \in N_{pa}$. For any $a \in N_{pa}$, we say that

- a is true, if $a \in I$,
 - a is false, if $\neg a \in I$, or
 - a is undefined (or unknown), if neither $a \in I$ nor $\neg a \in I$.
-
- Logic Programs describe mappings from (input) interpretation to (output) interpretation.

The Well-Founded Semantics of Logic Programs

- Three valued semantics: literals can be true, false, or *undefined*.
- Positive literals are determined by taking ground program, with facts and rules, and computing the maximal set of consequences.
- Negative literals are computed by computing the **maximal unfounded set**.
- Any ground literals not found in either of these two categories, is undefined.

Unfounded Set

A set $U \subseteq 2^{N_{pa}}$ is called an *unfounded set* w.r.t. an interpretation I and a LP P , if at least one of the following holds for each rule $\rho \in P$ with $head(\rho) \in U$:

- there is an atom $b \in body(\rho)$ s.t. $b \in U$ or $\neg b \in I$, or
 - there is a negative literal $\neg b \in body(\rho)$ s.t. $b \in I$.
-
- Intuition: an unfounded set captures atoms that can be “safely” negated.
 - **Complexity:** Evaluation for WFS possible in quadratic time⁴.

⁴The Well-Founded Semantics for General Logic Programs., A. Van Gelder, K. A. Ross, J. S. Schlipf, J.ACM 1991

Advantages of WFS for SHACL

- **Inconsistency tolerant**
no total assignment required to check target specification.

Advantages of WFS for SHACL

- **Inconsistency tolerant**
no total assignment required to check target specification.
- **No self supports**
such cases are avoided and assigned an unknown value.

Advantages of WFS for SHACL

- **Inconsistency tolerant**
no total assignment required to check target specification.
- **No self supports**
such cases are avoided and assigned an unknown value.
- **Scalability:**
Based on the complexity of WFS in the LP setting, SHACL under WFS is the most promising approach to have SHACL with full recursion with efficient validation.

Our Contribution

- We introduce a formal semantics for SHACL that builds on the Well-Founded Semantics of Logic Programs and covers all of SHACL Core⁵.
- We explain how, based on the formal semantics, we can find an efficient translation of SHACL under WFS into logic programs, hence utilising existing work on logic programming solvers.
- We finally showcase the practical potential of this translation by providing a prototype SHACL validator and comparing its performance against the state of the art.

⁵and could be easily extended to support SHACL-Sparql too

Implementation

- We wrote a prototype validator for SHACL under WFS (with a number of optimisations, allowing polynomial number of rules in the logic program).
- Our validator, written in Go , is called **shaWell**. Available here github.com/cem-okulmus/shawell/.
- It accesses the data via a SPARQL endpoint to either directly evaluate the SHACL document (in case of non-recursive documents), or
 - a) translates a given SHACL instance into a logic program (in case of recursion)
 - b) and this logic program is evaluated using DLV.

Experimental Evaluation

- Test instances (C1, C2, rec) of recursive SHACL documents from previous literature^{6 7}.
- Comparison with 3 other validators that support recursive SHACL.
- Timeout of 15 minutes, all numbers in seconds.

Test Set	Size	shaWell		SHaclEX ⁸		Shacl-Sparql ⁶		Apache Jena	
		TOs	med	TOs	med	TOs	med	TOs	med
C1	20	1	1.48	7	3.98	Fails to parse		20	-
C2	20	3	1.42	7	3.96	Fails to parse		20	-
rec	3	0	195.15	3	-	0	0.25	3	-

⁶*Validating Shacl Constraints over a Sparql Endpoint*, J. Corman, F. Florenzano, J.L.Reutter, O. Savkovic, ISWC 2019

⁷*Magic Shapes for SHACL Validation*, S. Ahmetaj, B. Löhnert, M. Ortiz, M. Šimkus, VLDB 2022

⁸Developed by Jose Emilio Labra Gayo, Professor at the University of Oviedo, Spain

Conclusion & Future work

- We presented a new semantics for the recursive fragment of SHACL.
- We defend the need for yet another semantics by showing how
 - ▶ SHACL under WFS behaves semantically well; and
 - ▶ that SHACL under WFS allows tractable validation.
- We provide a proof-of-concept implementation of SHACL under WFS, which implements the entire SHACL Core standard.

Future Work

- Use SHACL under WFS to provide a richer semantics for validation reports, aiming to serve as more useful explanations for validation failures.
- Use SHACL under WFS to enable repair suggestions in case of validation failure.
- There has been recent theoretical work on combining SHACL and ontologies⁹. SHACL under WFS seems a promising candidate to serve as a start point to implement such a system.

⁹*Reconciling SHACL and Ontologies: Semantics and Validation via Rewriting*, S. Ahmetaj, M. Ortiz, A. Oudshoorn, M. Šimkus, ECAI 2023

Thanks for your Attention!
Any Questions?



UMEÅ UNIVERSITY

